

Frameworkitty

Webservices Framework

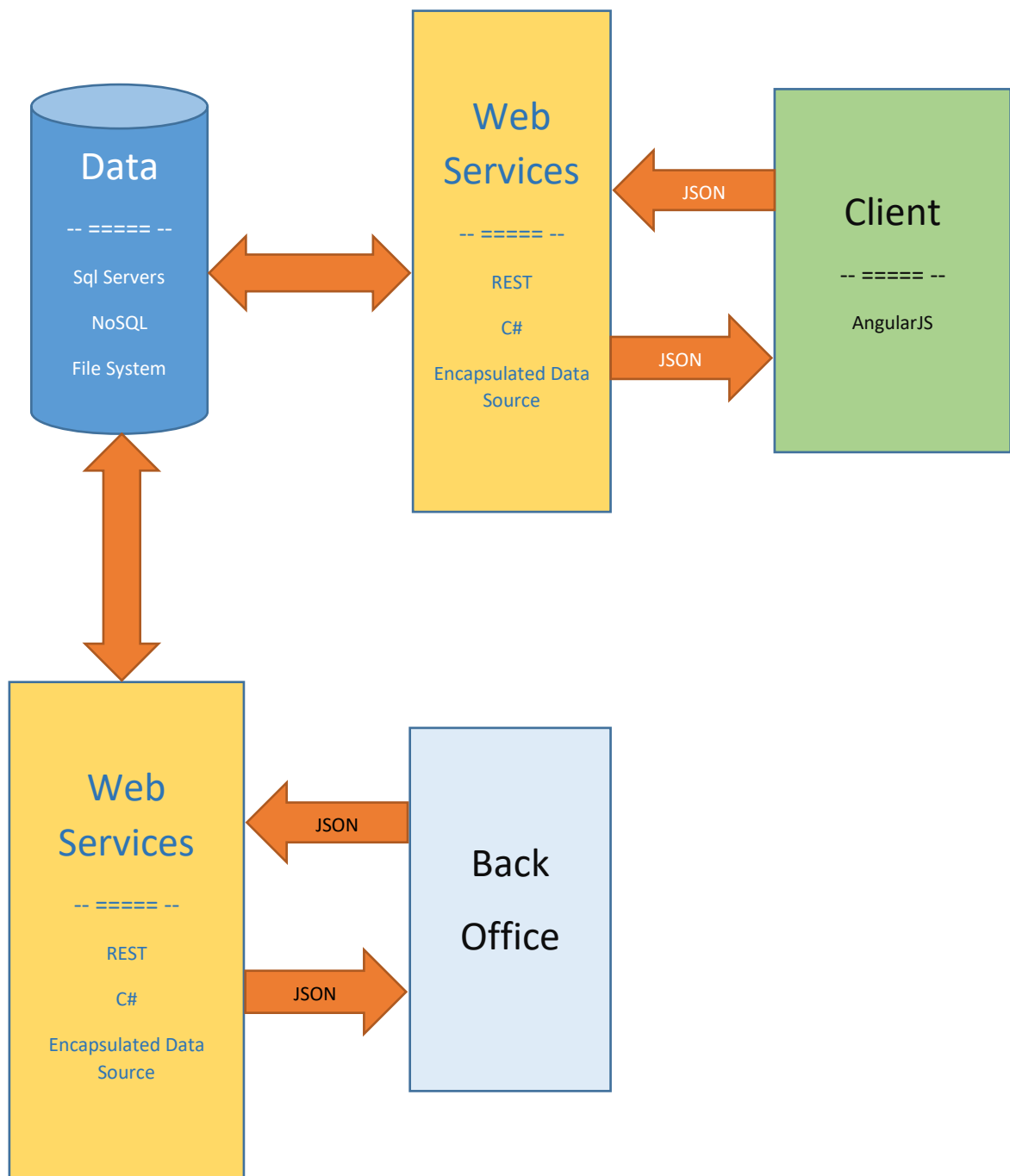
Histórico de Revisões

Data	Versão	Descrição	Autor(es)
18-08-2015	0.10	Criação e estruturação do documento.	Pedro Fernandes
24-08-2015	0.11	Modelo de Webservices	Pedro Fernandes
28-08-2015	0.12	Revisão	Pedro Fernandes
09-09-2015	0.20	Detalhe de Modelos JSON	Pedro Fernandes
11-09-2015	0.21	Exemplos de Permissões	Pedro Fernandes

Índice

Histórico de Revisões	3
Índice	4
Esquemático	5
Enquadramento	6
Funcionalidade	7
Acesso a Dados.....	8
DAO	8
OLE DB.....	9
Estrutura de Webservices	10
Modelo Genérico para Entidades (Back Office)	10
Modelo Genérico para Entidades (Front Office)	11
Conclusões	13
Modelo de Permissões	14
Descrição	15
Estrutura de Webservices	16
Exemplos para Entidades Genéricas	16
Exemplos específicos para Users	19
Considerações Finais	21
Appendix A	22
Modelos.....	22
Modelo JSON “books” versão “simple”	23
Modelo JSON “books” versão “books libraries”	24
Modelo JSON “books” versão “books libraries readers”	26
Appendix B	29
OLE DB.....	29
Appendix C	30
Permissões	30

Frameworkitty



Enquadramento

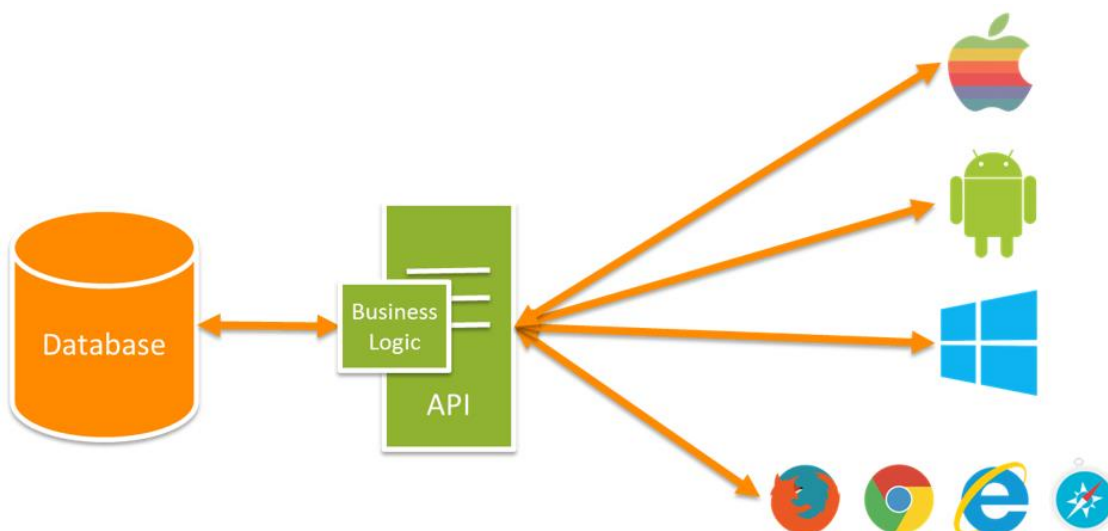
Este projecto consiste em criar, numa primeira fase, um aglomerado de conceitos a integrar numa framework que servirá para fornecer um conjunto de serviços a qualquer tipo de cliente, permitindo um isolamento total entre o cliente e a fonte de dados. Estes clientes poderão ser do tipo Front Office, onde os pedidos são sobretudo de manipulação de informação (input/output) para o utilizador final visualizar, mas também clientes de Back Office, onde os pedidos poderão ter outro significado, nomeadamente com a criação de estruturas para acesso a dados, remodelação das já existentes ou mesmo a sua remoção.

As operações que irão ser realizadas com os dados serão feita através de datasources (fontes de dados) que terão o comportamento adequado para aquilo que é pretendido que produzam. O acesso à informação é fornecido ao cliente do Front Office de forma transparente, consoante o planeamento do Back Office e a construção das estruturas necessárias para que tal ocorra, sendo que toda a comunicação com as fontes de dados será feita pela framework.

Autorização e autenticação, bem como um esquema de permissões que não vincule a framework a um modelo restrito, é também importante e praticamente um módulo à parte dos restantes conceitos que se pretende que sejam aplicados.

Módulos que contemplem gestão documental, um sistema de mensagens, tickets e suporte ao cliente bem como análise de dados, são adições e funcionalidades futuras, assim como a integração com CMS e sistemas de facturação.

Funcionalidade



Através da utilização de um modelo MVC para o cliente de Front Office (AngularJS por exemplo), serão feitos pedidos à API da Framewitty que disponibiliza os webservices necessários para esse efeito. Esses webservices irão comunicar com a Business Logic e com a Framework Logic onde estão incluídos os módulos de autenticação e autorização.

Na Framework Logic e após um pedido de autenticação oriundo dos webservices, esse pedido vai originar um acesso à fonte de dados para troca de informação, mas também um pedido ao módulo de autorização que comunicará com a Business Logic e esta por sua vez com a fonte de dados ou com uma camada de abstracção como podemos ver mais à frente.

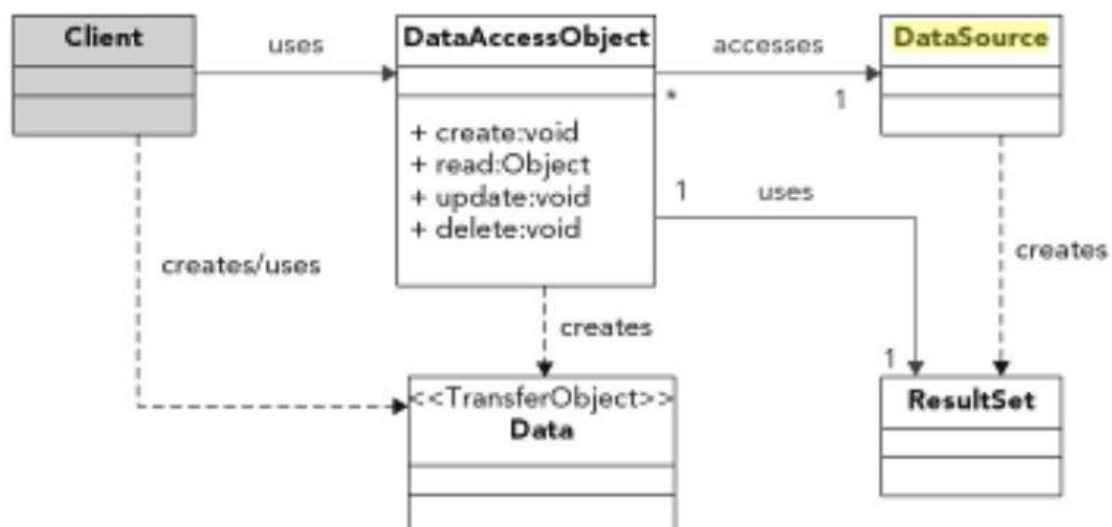
Esta API da Framework poderá igualmente ser chamada a partir de um CMS como por exemplo o Joomla, CMS esse que terá de disponibilizar a sua própria API para acesso a partir do cliente de Front Office, sendo o processo obrigatoriamente transparente para o utilizador.

Acesso a Dados

DAO

É algo impensável imaginarmos uma aplicação empresarial nos nossos dias em que não exista de alguma forma um acesso a dados, estejam eles guardados numa base de dados relacional, orientada a objectos ou NoSQL, bem como noutro tipo de formato como por exemplo um sistema de ficheiros, um webservice, um repositório LDAP (Lightweight Directory Access Protocol) ou um qualquer sistema externo.

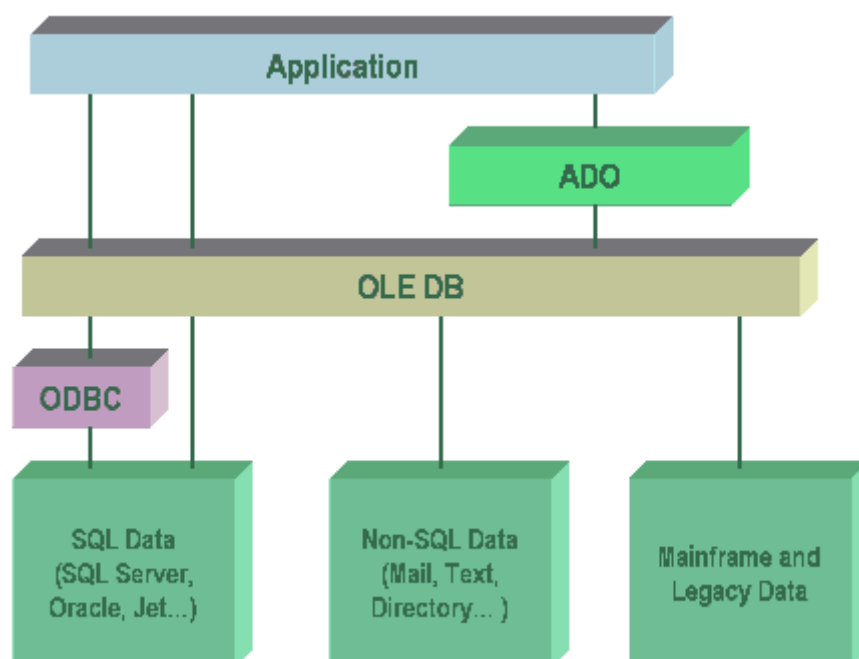
Uma das possibilidades de construção de um sistema em que se tenta ser o mais transparente possível, é através da criação de uma camada de abstracção para podermos separar claramente a Business Logic do código que acede a essa fonte dados, independentemente da forma como estão armazenados.



No exemplo da figura, e embora o ambiente Visual Studio .NET, Visual C++ .NET e os wizards tenham acabado com o suporte ao DAO (se bem que as classes ainda estejam incluídas e possam ser utilizadas, a Microsoft recomenda OLE DB Templates ou ODBC para novos projectos), um objecto DAO (Data Access Object) providencia um interface abstracto para que seja possível haver um encapsulamento no acesso a um qualquer tipo de fonte de dados ou outro mecanismo persistente, efectuando-se operações nesses dados (CRUD) sem ser necessário revelar detalhes.

Acrescentando a nossa camada de serviços à imagem da figura, que é aquilo a que o cliente realmente tem acesso, conseguimos aumentar a segurança do modelo e obter a tal separação desejada entre a Business Logic e o código de acesso a dados, assegurando que caso haja uma mudança na sua estrutura, o impacto na camada referida é praticamente nulo, ou pelo menos muito atenuado.

OLE DB



Criada pela Microsoft, a tecnologia OLE (*Object Linking and Embedding*) DB é um conjunto de interfaces COM (*Component Object Model*) que providenciam acesso uniforme a dados nos mais diversos formatos e origens. As OLE DB templates oferecem classes que implementam as interfaces OLE DB mais comuns, facilitando assim a utilização desta tecnologia de elevado desempenho.

Equipado com a possibilidade de utilizar soluções Universal Data Access, que providencia um método comum para aceder a dados independentemente do formato em que estão guardados, isso implica a existência de duas características: distributed query (acesso uniforme a fontes de dados distribuídas) e Non-DBMS access (possibilidade de fazer com que fontes de dados Non-DBMS estejam acessíveis para aplicações de bases de dados).

Estrutura de Webservices

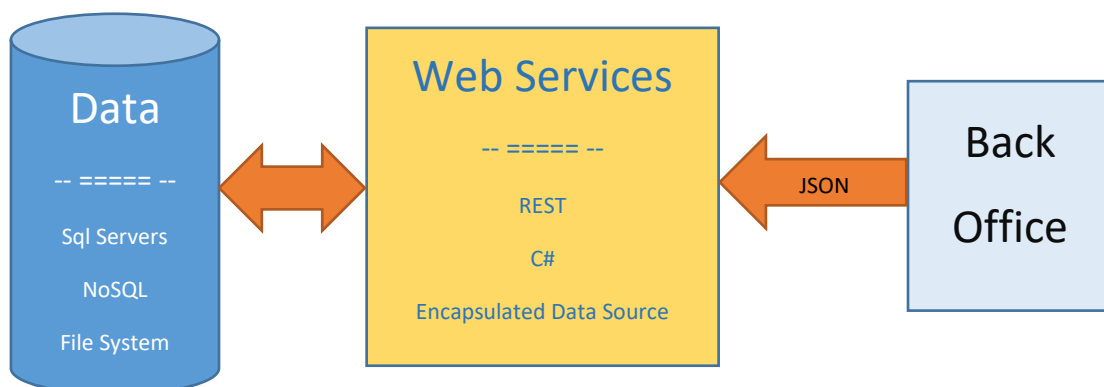
Modelo Genérico para Entidades (Back Office)

Através do Back Office, é possível desenhar-se um modelo de entidade e enviar-se para o webservice respectivo um JSON com toda a informação necessária para a criação da entidade no formato pretendido. Na sua versão regular (existem 3 versões para cada entidade, como poderemos ver mais à frente: compacta, regular e completa), esta estrutura deverá incluir, não sendo uma lista exaustiva e com alguns dos seus componentes opcionais, a seguinte informação:

Versão Regular

- Designação da Entidade
- Formato da fonte de dados em que a estrutura será guardada
- Dados de acesso à fonte dados
- Tipo de operação a realizar na fonte dados
- Lista de “campos” com informação de funcionalidade, tipo e tamanho, podendo-se incluir outros tipos de propriedades, nomeadamente de validação

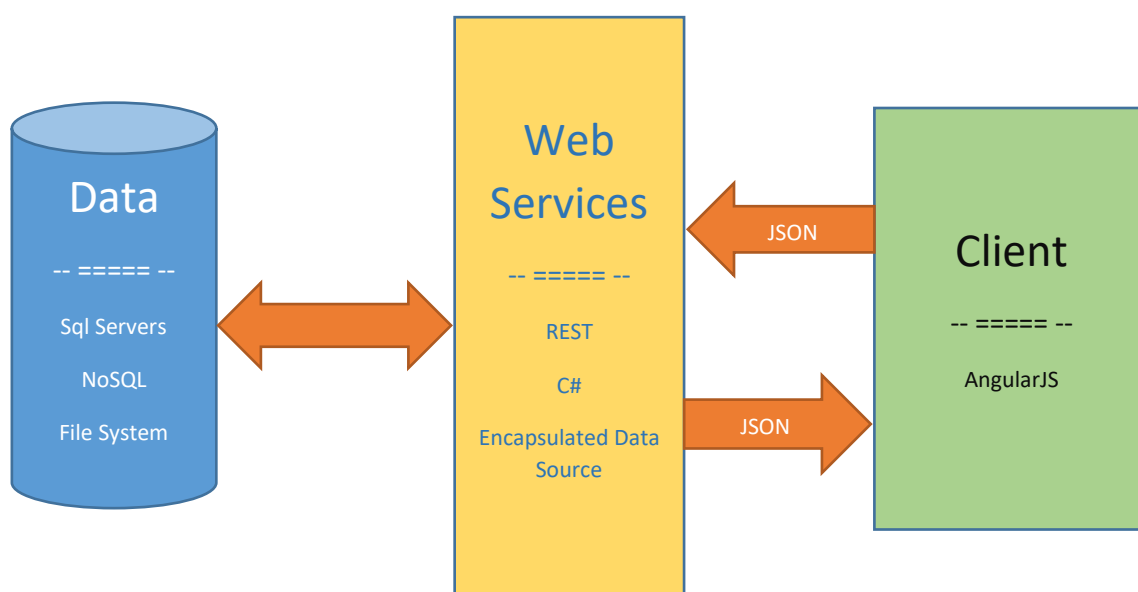
Na versão compacta, apenas se incluirão os campos que são realmente necessários para se obter uma informação rápida enquanto que na versão completa se incluem todos os campos da entidade, tal como na versão regular, mas também informação sobre outras estruturas que estejam relacionadas e das quais seja necessário extrair/inserir informação, como por exemplo o role e relations para uma entidade USERS.



Após a recepção do pedido com a estrutura em JSON, o webservice irá determinar qual a fonte de dados que vai utilizar, executar uma operação do tipo pretendido e guardar a informação sobre a entidade numa fonte de dados autónoma que servirá para mais tarde consultar, aquando dos pedidos por parte do Front Office.

Modelo Genérico para Entidades (Front Office)

No que diz respeito ao Front Office, e através de utilização de um URL do tipo `https://api/v1/entities` podemos executar as operações que pretendemos para uma determinada entidade, incluindo o seu nome na informação que enviamos. Mais uma vez temos três versões diferentes, compacta, regular e completa, que poderemos diferenciar através de informação enviada ou através do método de acrescentar ao URL a versão pretendida, caso não seja a regular.



Uma das vantagens deste método de funcionamento é que o URL a utilizar pelo Front Office é o mesmo e independente da entidade, evitando-se ter de alterar a API para fornecer serviços que respondam a pedidos semelhantes mas para entidades diferentes.

Se por exemplo quisermos criar um registo numa entidade chamada BOOKS e apenas pretendemos inserir os dados todos que existem na estrutura desse BOOKS, basta chamarmos o URL `https://api/v1/entities` com a indicação que a entidade é BOOKS, do tipo POST e com a informação respectiva que a API carrega o modelo de entidade BOOKS, previamente inserido pelo Back Office, faz match à informação recebida, cria o registo e devolve a estrutura que acabou de criar.

Num exemplo mais complexo, se quisermos criar um registo numa entidade BOOKS mas também acrescentar um registo a uma entidade chamada LIBRARIES e relacioná-los, visto ser um novo registo do tipo LIBRARIES e aquele ser o primeiro BOOKS que vai ser inserido, podemos chamar o URL <https://api/v1/entities/complete> referindo na informação enviada a estrutura dos dados a inserir, mais uma vez fazendo match com o modelo criado pelo Back Office e executando a operação de concretizar as duas inserções de uma só vez.

Num exemplo mais simples, se se pretender obter informação compactada (e rápida) sobre um determinado BOOKS, basta utilizar o URL https://api/v1/entities/ID_MODELS/compact com um pedido do tipo GET e com a entidade BOOKS devidamente identificada, e mais uma vez dependendo do modelo criado, apenas recebo rapidamente o ID_BOOKS e o Title (por exemplo).

Um dos aspectos a ter em conta no Front Office é a questão da paginação, pois dependendo da quantidade de informação devolvida se o pedido receber como resposta toda a informação solicitada, o tempo de execução poderá ser elevado. A questão da paginação será abordada noutra altura, nomeadamente a implementação de pedidos onde é enviada a informação do número máximo de registos pretendidos e qual o intervalo desejado, obtendo como resposta apenas esses mesmo registos, podendo o resto da informação ser parcialmente entregue em pedidos subsequentes.

Também a questão de caching não será aqui abordada de momento, embora seja um tópico que poderá ter impacto no funcionamento da solução. Segundo o princípio em que se uma solução for lenta convém adicionar caching e se tiver muitos bugs convém remover o caching, encontrar o balanceamento ideal entre a sua utilização não é por vezes uma tarefa fácil.

Conclusões

No fundo podemos ver as funcionalidades do sistema como o carregar de modelos de entidades da parte do Back Office, modelos esses que serão criados pela API no formato desejado de dados e com uma camada de isolamento, sendo guardados para futura referência, aquando da recepção de pedidos do Front Office. Quando esses pedidos chegam, é carregado o modelo pretendido na versão correcta, pois este vem identificado, e feito o match da informação de acordo com o modelo que foi guardado.

No exemplo acima mencionado consideramos a relação entre BOOKS e LIBRARIES de um para muitos, mas nada nos impede de fazermos o mesmo para relações de muitos para muitos, bastando que o modelo carregado seja coerente e contemple esse tipo de relação.

No limite podemos usar vários tipos de modelos para cada entidade e nomeá-los através do URL como no exemplo da página anterior, mas também utilizar o URL normal e enviar informação no pedido sobre o tipo de versão que queremos.

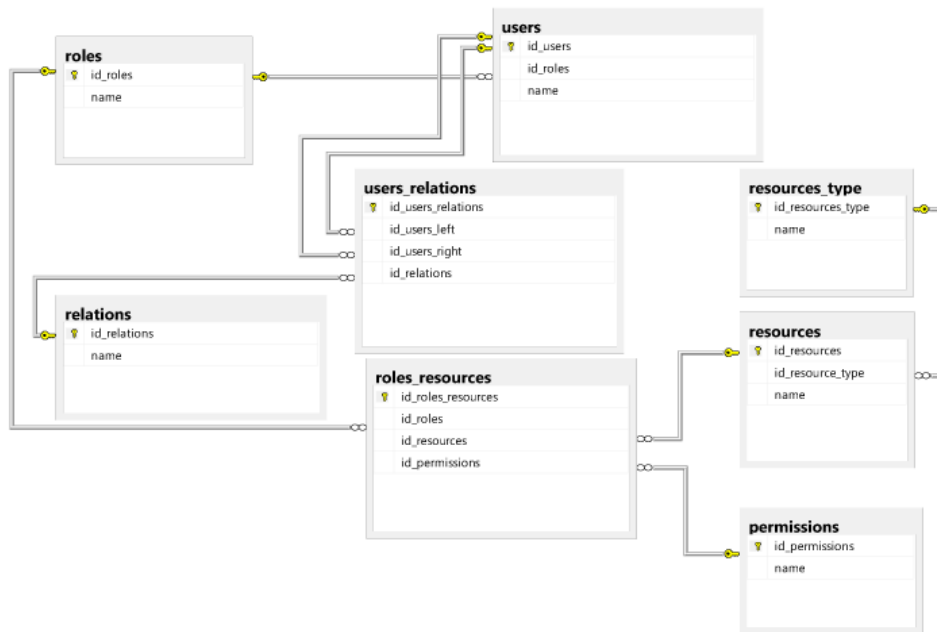
A passagem de campos como parâmetros no URL também é um método passível de ser utilizado em casos especiais nos quais não seja conveniente utilizar os modelos criados.

Em termos de desenvolvimento no Front Office e caso seja conveniente, em vez de se passar a informação sobre qual a entidade pretendida no pedido, o URL poderá incluir essa informação, tal como foi referido para o caso das versões.

O facto de podermos inserir todo um conjunto de dados em diversas entidades com um só pedido, apenas utilizando um dos modelos à disposição, é um dos factores mais importantes a ter em conta nas funcionalidades da Framewitty, pois o resultado a receber por quem faz o pedido é do tipo *Sucesso* ou *Insucesso*, com a informação adicional para cada caso. Este tipo de resposta assegura a integridade dos dados e que ou toda a informação é guardada como se pretende, ou nada foi guardado devido a uma qualquer falha, provavelmente identificada e descrita na resposta.

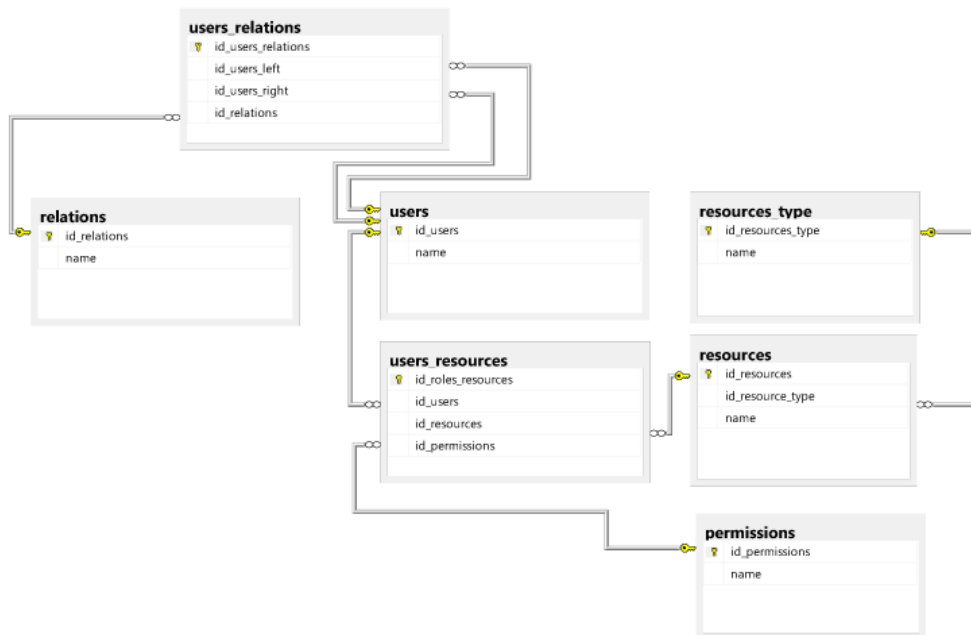
Modelo de Permissões

Role-Based Access Control (RBAC)



Discretionary Access Control (DAC)

Access Control Entry (ACE)



Descrição

Em termos de expansão, podemos complicar os esquemas criados se considerarmos que determinadas relações entre utilizadores influenciam as permissões que esse utilizador poderá ter em termos de acessos do role a determinadas resources (RBAC) ou acrescentar a necessidade de haver um determinado tipo de relação com outros utilizadores para ser possível ao utilizador fazer parte da lista de acesso a determinado resource (DAC).

A definição de utilizador da esquerda e da direita poderá ser usada para se ter uma noção de sentido, sobretudo se o esquema montado for do tipo hierárquico, onde poderá existir um pai e um filho em cada nó de uma árvore. Por outro lado também poderão ser usadas relações em que ambos os utilizadores estão ao mesmo nível, empregando-se mesmo estruturas onde um conjunto não vazio de utilizadores se relaciona através de um conjunto de pares não ordenados. Grafos não direccionados no caso de ausência de sentido ou se este existir, grafos orientados.

Os resources poderão ser de diferentes tipos e com uma granularidade de alto ou baixo nível, como por exemplo um item de menu, uma form, um URL ou um determinado campo numa base de dados. A existência de um `resource_value` a atribuir a cada resource também será algo a ter em consideração pois permite resolver casos em que apareçam conflitos, podendo assim serem estabelecidas prioridades.

Tal como existem diversos tipos de resources, também as permissões poderão ser ilimitadas. Para além das habituais ver, editar, inserir e apagar, podemos atribuir permissões de interface, consultar logs ou mesmo alterar estruturas.

Estrutura de Webservices

Exemplos para Entidades Genéricas

Num primeiro exemplo e para o caso das entidades genéricas, vamos utilizar a aproximação em que a identificação da entidade é feita como parâmetro do pedido. Mais à frente temos o exemplo particular dos USERS, que no fundo são entidades, mas em que a aproximação é diferente pois identifica-se a entidade através do próprio URL. Podem-se utilizar os dois métodos, dependendo sobretudo do esquema de trabalho a utilizar no Front End.

Modelo estrutural a utilizar para a entidade BOOKS:

	Compact	Regular	Complete
<i>ID_BOOKS</i>	v	v	v
<i>Title</i>	v	v	v
<i>Author</i>		v	v
<i>ID_LIBRARIES</i>			v
<i>Name</i>			v

Como podemos ver pela figura apresentada, dependendo do pedido efectuado, é possível receber como resposta uma estrutura Books Compact em que apenas os campos essenciais são incluídos, uma estrutura Books Regular que será o mais utilizado no funcionamento normal de operações CRUD, mas também uma estrutura Books Complete em que podemos receber de uma só vez todos os registos pretendidos que fazem parte da relação BOOKS - LIBRARIES e que queremos incluir num GET, ou mesmo inserir todos os registos da relação BOOKS - LIBRARIES de uma só vez com um POST, bastando para tal usar o modelo Books Complete e preencher adequadamente os valores que queremos inserir.

Este POST será efectuado de uma só vez e ou insere com sucesso todos os registos nos seus respectivos sítios, ou então falha, não se registando qualquer alteração na fonte de dados.

ENTITIES

<i>https://api/v1/entities</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>	<i>ID_MODELS</i>	Required	<i>MODELS Type</i>	List
<i>POST</i>	<i>Yes</i>	<i>ID_MODELS</i>	Required	<i>MODELS Type</i>	Just created
		<i>Model Key</i>	Optional		
		<i>Model Field1</i>	Required		
		<i>Model Field2</i>	Optional		

<i>https://api/v1/entities/KEY</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>	<i>ID_MODELS</i>	Required	<i>MODELS Type</i>	One Structure
	<i>Yes/No</i>	<i>ID_BOOKS</i>	Required		
<i>PUT</i>	<i>Yes</i>	<i>ID_MODELS</i>	Required	<i>MODELS Type</i>	Just Updated
		<i>Model Key</i>	Required		
		<i>Model Field1</i>	Optional		
		<i>Model Field2</i>	Optional		
<i>DELETE</i>	<i>Yes</i>	<i>ID_MODELS</i>	Required	<i>MODELS Type</i>	Just Deleted
		<i>Model Key</i>	Required		

<i>https://api/v1/entities/compact</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>	<i>ID_MODELS</i>	Required	<i>MODELS Compact</i>	List

<i>https://api/v1/entities/complete</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>	<i>ID_MODELS</i>	Required	<i>MODELS Complete</i>	One Structure
<i>POST</i>	<i>Yes</i>	<i>ID_MODELS</i>	Required	<i>MODELS Complete</i>	Just created
		<i>Model Key</i>	Optional		
		<i>Model Field1</i>	Required		
		<i>Model Field2</i>	Optional		
		<i>ID_LIBRARIES</i>	Optional		
		<i>Name</i>	Optional		

Um aspecto que poderá por vezes ser ligeiramente incómodo, tem a ver com o facto de se enviar a identificação da identidade (qual o modelo a utilizar) como parâmetro, o que obrigaria a alterar um GET em que habitualmente não precisaríamos de enviar parametros, havendo nesse caso a necessidade de se criar mais código.

Por outro lado, podemos enviar de uma vez só o que seriam vários pedidos para URLs diferentes, se a identificação das entidades estivesse presente nesse mesmo URL. Utilizando o método aqui apresentado, através de um pedido a *https://api/v1/entities* podemos fazer inserções em diversas tabelas sem qualquer ligação entre si, bastando apenas estarem identificados nos parâmetros quais os modelos a utilizar. Isto poderá também ser útil em situações de sincronização ou naquelas em que não existe ligação à internet.

Nos exemplos seguintes, e para se poder ter em conta métodos alternativos, usamos aquele em que a identificação da entidade está presente no URL.

Exemplos específicos para Users

Neste caso específico de USERS, vamos dar alguns exemplos de webservice a partir de uma estrutura que contempla apenas uma relação de um para muitos entre USERS e ROLES. Vamos definir quais os três modelos que referimos acima, tendo no entanto a percepção que pode existir um número ilimitado de modelos e que a designação do modelo a utilizar pode ser passada como parâmetro e não no URL como nos exemplos apresentados.

Modelo estrutural a utilizar:

	Compact	Regular	Complete
<i>ID_USERS</i>	v	v	v
<i>Name</i>	v	v	v
<i>Address</i>		v	v
<i>ID_ROLES</i>			v
<i>Roles Name</i>			v

Como podemos ver pela figura apresentada, dependendo do pedido efectuado, é possível receber como resposta uma estrutura Users Compact em que apenas os campos essenciais são incluídos, uma estrutura Users Regular que será o mais utilizado no funcionamento normal de operações CRUD, mas também uma estrutura Users Complete em que podemos receber de uma só vez todos os registos pretendidos que fazem parte da relação USERS - ROLES e que queremos incluir num GET, ou mesmo inserir todos os registos da relação USERS - ROLES de uma só vez com um POST, bastando para tal usar o modelo Users Complete e preencher adequadamente os valores que queremos inserir.

Este POST será efectuado de uma só vez e ou insere com sucesso todos os registos nos seus respectivos sítios, ou então falha, não se registando qualquer alteração na fonte de dados.

USERS

<i>https://api/v1/entities/USERS</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>			<i>USERS</i>	List
<i>POST</i>	<i>Yes</i>	<i>ID_USERS</i>	Optional	<i>USERS</i>	Just created
		<i>Name</i>	Required		
		<i>Address</i>	Optional		

<i>https://api/v1/entities/USERS/ID_USER</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>	<i>ID_USERS</i>	Required	<i>USERS</i>	One Structure
<i>PUT</i>	<i>Yes</i>	<i>ID_USERS</i>	Required	<i>USERS</i>	Just Updated
		<i>Name</i>	Optional		
		<i>Address</i>	Optional		
<i>DELETE</i>	<i>Yes</i>	<i>ID_USERS</i>	Required	<i>USERS</i>	Just Deleted

<i>https://api/v1/entities/USERS/compact</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>			<i>USERS Compact</i>	List

<i>https://api/v1/entities/USERS/complete</i>					
http Method	Logged	Parameters	Properties	Response	Obs
<i>GET</i>	<i>Yes/No</i>			<i>USERS Complete</i>	One Structure
<i>POST</i>	<i>Yes</i>	<i>ID_USERS</i>	Optional	<i>USERS Complete</i>	Just created
		<i>Name</i>	Required		
		<i>Address</i>	Optional		
		<i>ID_ROLES</i>	Optional		
		<i>Roles Name</i>	Optional		

Considerações Finais

A Framework aqui apresentada tem como um dos seus objectivos o isolar as fontes de dados dos clientes que pretendem consultar ou inserir informação, mas sobretudo ser versátil no aspecto de modelização e permitir crescer em termos de funcionalidades.

Através da inserção de modelos para posterior consulta e match making, consegue-se criar uma framework que no limite não precisa de ser alterada de projecto para projecto, ou quando se acrescenta algo não previsto ao projecto corrente, bastando introduzir informação sobre as alterações.

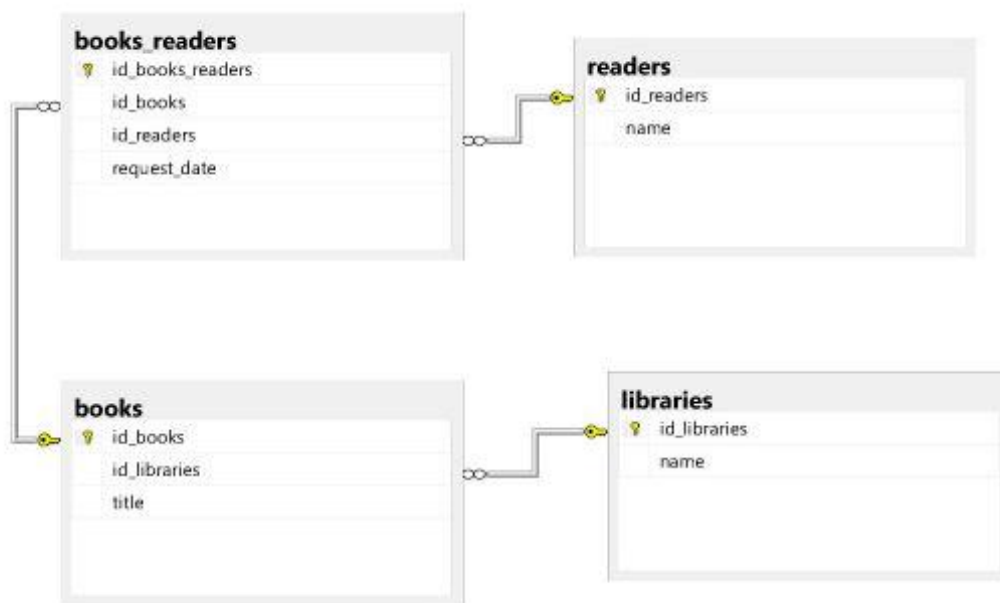
Um dos aspectos por onde poderá evoluir será na inserção de informação nos modelos a carregar pelo Back Office, nomeadamente no que diz respeito a informação sobre operações a realizar, do tipo haver um campo que vai ser multiplicado por outro, podendo assim o Front Office obter dados estatísticos, por exemplo, sem ser preciso uma vez mais alterar a API.

Para operações mais complexas, pode-se mesmo criar um motor de regras para a API que servirá para a “educar” no tipo de operações que tiver de realizar, sejam elas quais forem, de forma a que quando a API receber um pedido do Front Office para utilizar um determinado modelo de dados, a API consiga reconhecer que tem operações a fazer nos dados e saiba como as executar através de uma consulta a uma fonte de dados onde estarão as regras carregadas pelo Back Office do motor de regras.

###

Appendix A

Modelos



O carregamento de modelos será feito através do Back Office, como foi explicado anteriormente. Aproveitando a relação BOOKS - LIBRARIES, incluindo a entidade READERS e uma relação muitos para muitos entre BOOKS e READERS que dá origem à tabela BOOKS_READERS como se pode ver na figura apresentada acima, construímos vários modelos JSON que servem de exemplo para ilustrar o modo como podem ser introduzidos no Framewitty para depois serem chamados pelo Front Office, fazendo match à informação recebida.

De notar que a informação que vai em “operation” serve para depois do Back Office enviar o pedido e armazenarmos o modelo na tabela MODELS (*id_models, name, version, json_structure, date_created, date_updated*) , saber o que fazer com a estrutura, nomeadamente que tabelas criar.

Modelo JSON “books” versão “simple”

Eis o JSON do modelo “books” versão “simple”:

```
{
  "books": [
    {
      "table": "books",
      "version": "simple",
      "operation": "info"
    },
    {
      "id_books": [
        {
          "type": "int",
          "size": 10,
          "validation": "not null"
        }
      ],
      "id_libraries": [
        {
          "type": "int",
          "size": 10,
          "validation": "not null"
        }
      ],
      "title": [
        {
          "type": "varchar",
          "size": 100,
          "validation": "not null"
        }
      ]
    }
  ]
}
```

Modelo JSON “books” versão “books libraries”

Se quisermos complicar o modelo do exemplo anterior e criar um outro que inclui a relação um para muitos entre BOOKS - LIBRARIES, poderíamos carregar o JSON do seguinte modelo “books” versão “books libraries”:

```
{
  "books": [
    {
      "table": "books",
      "version": "books libraries",
      "operation": "create"
    },
    {
      "id_books": [
        {
          "type": "int",
          "size": 10,
          "validation": "not null"
        }
      ],
      "id_libraries": [
        {
          "type": "int",
          "size": 10,
          "validation": "not null",
          "link_table": [
            {
              "libraries": [
                {
                  "id_libraries": [
                    {
                      "type": "int",
                      "size": 10,
                      "validation": "not null"
                    }
                  ],
                  "name": [
                    {
                      "type": "varchar",
                      "size": 100,
                      "validation": "not null"
                    }
                  ]
                }
              ]
            }
          ],
          "operation": "create"
        }
      ]
    }
  ]
}
```

```
    ]  
  }  
],  
"title": [  
  {  
    "type": "varchar",  
    "size": 100,  
    "validation": "not null"  
  }  
]  
}  
]
```

Modelo JSON “books” versão “books libraries readers”

Finalmente e dando exemplos para os diversos casos, decidimos incluir neste último modelo a tabela READERS e criar o JSON do modelo mais complexo de “books”, a versão “books libraries readers”:

```
{
  "books": [
    {
      "table": "books",
      "version": "books libraries readers",
      "operation": "info"
    },
    {
      "id_books": [
        {
          "type": "int",
          "size": 10,
          "validation": "not null",
          "link_table": [
            {
              "books_readers": [
                {
                  "id_books": [
                    {
                      "type": "int",
                      "size": 10,
                      "validation": "not null"
                    }
                  ],
                  "id_books_readers": [
                    {
                      "type": "int",
                      "size": 10,
                      "validation": "not null"
                    }
                  ],
                  "id_readers": [
                    {
                      "type": "int",
                      "size": 10,
                      "validation": "not null",
                      "link_table": [
                        {
                          "readers": [
                            {
                              "id_readers": [
                                {
```

```

        "type": "int",
        "size": 10,
        "validation": "not null"
    }
],
    "name": [
        {
            "type": "varchar",
            "size": 100,
            "validation": "not null"
        }
    ]
}
],
    "operation": "info"
}
]
}
],
    "request_date": [
        {
            "type": "datetime",
            "size": 19,
            "validation": "not null"
        }
    ]
}
],
    "operation": "info"
}
]
}
],
    "id_libraries": [
        {
            "type": "int",
            "size": 10,
            "validation": "not null",
            "link_table": [
                {
                    "libraries": [
                        {
                            "id_libraries": [
                                {
                                    "type": "int",
                                    "size": 10,
                                    "validation": "not null"
                                }
                            ],
                            "name": [

```

```

        {
            "type": "varchar",
            "size": 100,
            "validation": "not null"
        }
    ]
}
],
"operation": "create"
}
]
}
],
"title": [
    {
        "type": "varchar",
        "size": 100,
        "validation": "not null"
    }
]
}
]
}

```

Appendix B

OLE DB

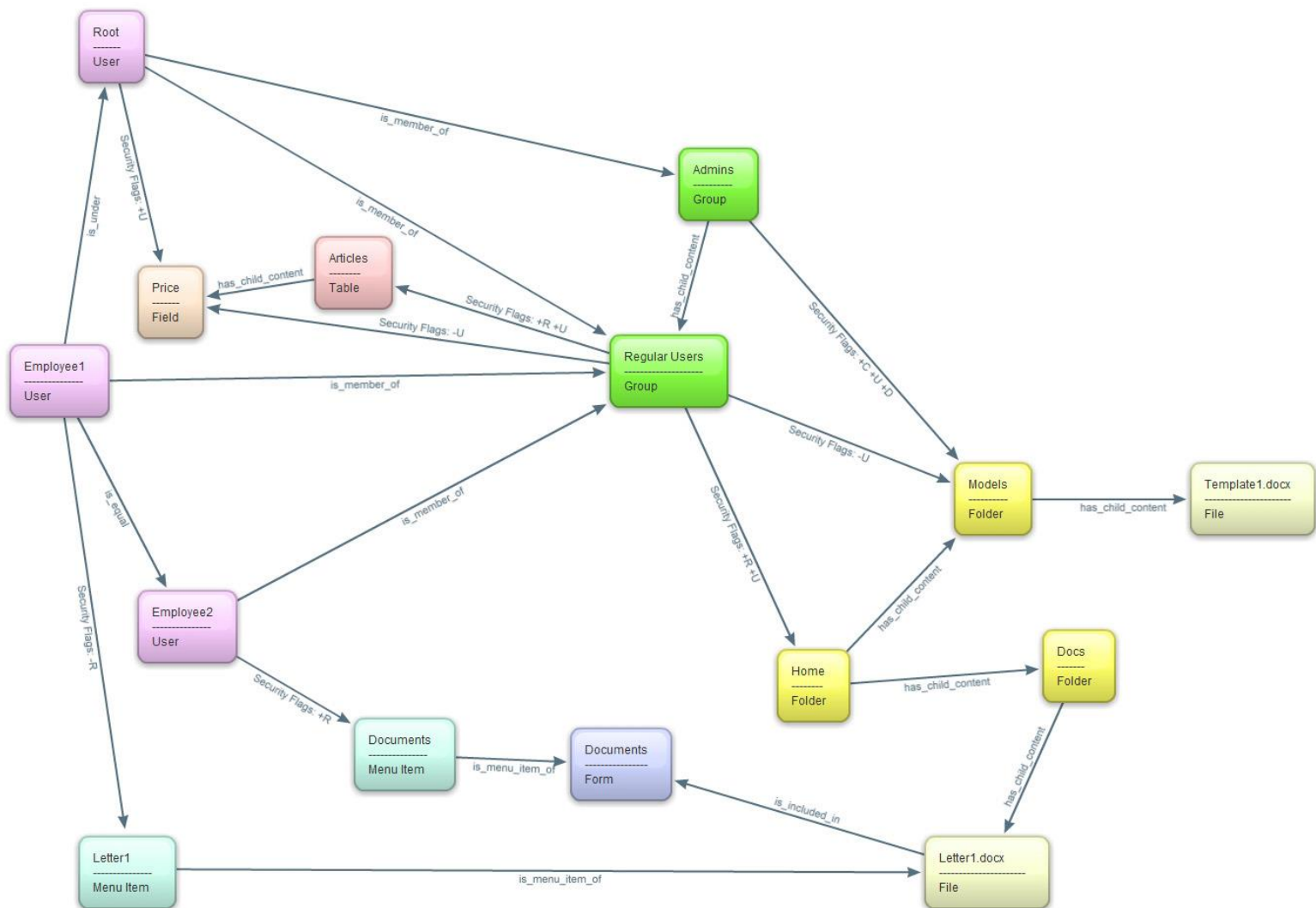
Lista de OLE DB Providers:

- OLE DB Provider for Active Directory Service
- OLE DB Provider for Advantage
- OLE DB Provider for AS/400 (from IBM)
- OLE DB Provider for AS/400 and VSAM (from Microsoft)
- OLE DB Provider for Commerce Server
- OLE DB Provider for DB2
- OLE DB Provider for DTS Packages
- OLE DB Provider for Exchange
- OLE DB Provider for Excel
- OLD DB Provider for Internet Publishing
- OLE DB Provider for Index Server
- OLE DB Provider for Microsoft Jet
- OLE DB Provider for Microsoft Project
- OLE DB Provider for MySQL
- OLE DB Provider for ODBC Databases
- OLE DB Provider for OLAP Services
- OLE DB Provider for Oracle (from Microsoft)
- OLE DB Provider for Oracle (from Oracle)
- OLE DB Provider for Pervasive
- OLE DB Provider for Simple Provider
- OLE DB Provider for SQLBase
- OLE DB Provider for SQL Server
- OLE DB Provider for SQL Server via SQLXMLOLEDB
- OLE DB Provider for Sybase Adaptive Server Anywhere
- OLE DB Provider for Sybase Adaptive Server Enterprise
- OLE DB Provider for Text Files
- OLE DB Provider for UniData and UniVerse
- OLE DB Provider for Visual FoxPro

Appendix C

Permissões

Tendo por base o capítulo modelo de permissões e as considerações lá introduzidas, acrescentando-se a possibilidade de haver relações entre users – resources e igualmente de resources - resources, criou-se a figura indicada que é um mero exemplo de representação de relações, para uma situação em que temos apenas permissões CRUD relacionadas com a segurança.



Neste exemplo, e se considerarmos uma aproximação top – down, podemos tirar algumas conclusões no que diz respeito às permissões dos utilizadores no acesso a determinadas resources, e qual a posição hierárquica entre eles.

Começando com o acesso ao campo *Price* da tabela *Articles*, podemos ver que tanto o *Employee1* como o *Employee2* têm apenas acesso +R, visto ambos pertencerem ao grupo *Regular Users*. Recebem o acesso +R +U na tabela *Articles*, mas depois o acesso U ao campo *Price* é-lhes removido. Já o *Root* tem acesso +R +U, pois embora pertencendo também ao mesmo grupo que lhe dá acesso +R, e visto este ser mais genérico, beneficia da permissão +U da ligação directa ao *Price* que se sobrepõe.

O *Employee1* tem acesso +R +U ao ficheiro *Letter1.docx*, graças ao caminho percorrido desde o grupo *Regular Users* até ao ficheiro, através dos folders *Home* e *Docs*, mas se estivéssemos numa aplicação em que o único acesso ao documento seria através de itens de menu, não o conseguiria ver devido ao acesso R lhe ser retirado no item de menu *Letter1*.

Já o *Employee2* conseguiria editar o ficheiro, pois embora não tenha qualquer relação com o item de menu *Letter1*, seguindo o top – down, tem acesso +R ao menu item *Articles* e consequente ao form *Articles*, mas as permissões ao ficheiro *Letter1.docx*, que está incluído no form, são +R +U devido ao grupo a que *Employee2* pertence.

Estes são apenas alguns exemplos do que se pode fazer com este tipo de representações, sendo no entanto necessário ter-se o maior cuidado na sua criação, pois facilmente se poderá entrar em casos de redundância ou mesmo contraditórios.